

Exact architecture exploration for heterogeneous edge–cloud cyber-physical systems

Nikita Tarasov^a, Vasyl Tomyuk^a

^a*Lviv Polytechnic National University, Lviv, Ukraine*

Abstract

Deciding how many edge nodes and gateways a distributed cyber-physical system should have, which hardware and network classes to use, where each processing stage runs and how far it is replicated determines whether the system meets its deadline and what it costs to run. The number of candidate architectures grows into the millions, while a single candidate is evaluated by simulation that can take up to a minute, so the practical budget is the number of expensive evaluations an engineer can afford. Exhaustive search pays for every candidate; evolutionary search fits a budget but cannot say which trade-offs it missed; and existing exact methods assume objectives that accumulate monotonically with each decision, which end-to-end latency does not, because it depends on the utilisation of a tier that several decisions fix jointly. Here we show that exact exploration remains achievable for such objectives. Bounds obtained by relaxing each term of an objective separately, combined with a decision order that makes the queueing term exactly computable on incomplete designs, allow provably safe elimination of infeasible and dominated regions before any evaluation. On spaces of up to 8.3×10^7 architectures the method calls the expensive evaluator 16–61 times and returns the exact set of optimal trade-offs wherever exhaustive enumeration can confirm it. In an independent containerised deployment with the model frozen beforehand, the predicted ordering matches the measured ordering for 88.6–91.3% of clearly separated architecture pairs.

Keywords: architecture exploration, cyber-physical systems, edge computing, multi-objective optimisation, design automation

Introduction

Engineering a distributed cyber-physical system is an architecture decision before it is a software decision. How many edge nodes and gateways to deploy, which hardware class each should be, where each processing stage runs, how far the ingest stage is replicated and which class of network link connects the tiers together determine whether the system meets its deadline, what it costs, how much energy it draws and how much traffic it puts on the network. These decisions interact: adding edge nodes shortens queues but raises cost and idle power, moving analytics to the cloud unloads the edge but adds a wide-area transfer to every request, and replication buys ingest throughput at the price of compute load and synchronisation traffic.

Email address: `dev.nikita@outlook.com` (Nikita Tarasov)

The number of candidate architectures is the product of the decision domains and reaches 10^5 – 10^7 for even a modest set of variables. What makes this different from classical processor-level design-space exploration is not the arithmetic but the price of one answer: for distributed cyber-physical systems a single design point is evaluated by simulation, and an industrial methodology built on execution traces of a lithography machine reports up to a minute per design point while explicitly leaving the search strategy out of scope[1]. A survey of the area concludes that scalable exploration technology for such systems is "more or less non-existing"[2]. The engineering consequence is concrete: the budget that matters is the number of expensive evaluations an engineer can afford, not the size of the space.

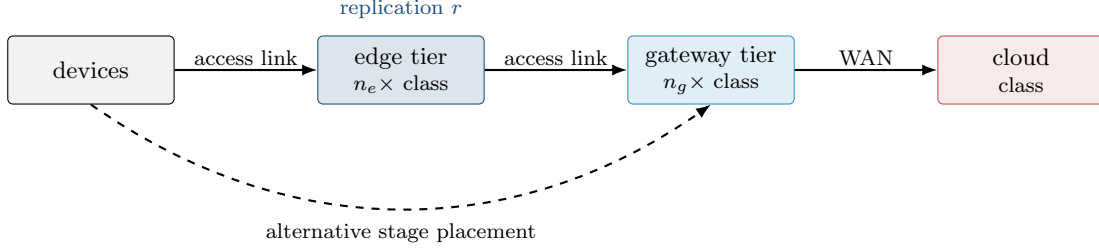
Two established options each give up something. Exhaustive enumeration returns the exact set of optimal trade-offs but pays the evaluator for every candidate. Population-based search such as NSGA-II[3] fits a fixed evaluation budget but returns an approximation and, crucially for an engineering decision, cannot say which trade-offs it missed. A third and smaller family reasons about *partial* designs and prunes regions that provably contain nothing optimal, either symbolically with constraint solvers[4, 5] or through bound-based Pareto pruning in the wider optimisation literature[6, 7]. Integer programming achieves the same guarantee for allocation problems in the edge-cloud continuum[8, 9].

Every method in that third family rests on one assumption: that each decision can only add to the objective, so that a partially specified design already carries a valid lower bound on all its completions. The dominant objective in a distributed cyber-physical system violates it. End-to-end latency contains a waiting term that depends on the utilisation of a tier, and utilisation is fixed only jointly, by the number of nodes, their class, the replication factor and the placement policy at once. Adding a node lowers it. There is therefore no monotone accumulation to bound, and the guarantee that makes exact pruning attractive does not transfer.

Here we show that exact exploration is nevertheless achievable for these objectives, and that it makes expensive architecture evaluation tractable rather than merely faster. We construct admissible bounds for contention-dependent objectives by relaxing each additive term of an objective independently over the domains still reachable, and by ordering the decisions so that the determinants of tier utilisation are fixed early — at which point the waiting term stops being relaxed and becomes exactly computable on an incomplete design. Pruning built on these bounds is provably safe, and the exploration returns the exact Pareto front of the analytical model.

The engineering payoff is a change of regime rather than a constant factor. On architecture spaces of up to 8.3×10^7 candidates the method expands at most 16 780 partial designs and calls the expensive evaluator 16–61 times, while the front it returns is identical, vector for vector, to the one exhaustive enumeration returns wherever exhaustive enumeration is still feasible. An independent containerised edge-gateway-cloud deployment, run with the analytical model frozen beforehand, shows that the ordering the model produces is the ordering the hardware produces for 88.6–91.3% of the architecture pairs it separates by more than 25% within a workload. We report equally clearly where the approach does not help: when the evaluator is cheap, when no admissible bound can be derived, and near saturation, where neither the model nor the deployment is stable.

a



b

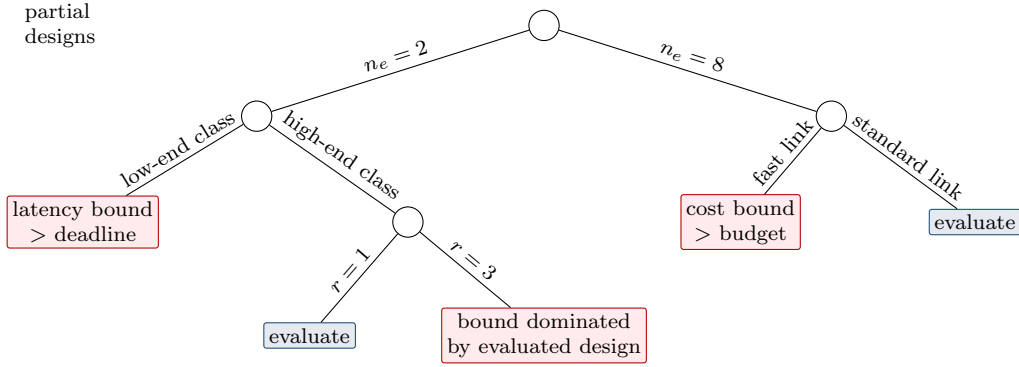


Figure 1: **Exploration by elimination.** **a**, Architecture template: the number and class of edge nodes and gateways, the class of the cloud instance, the access link class, the replication factor of the ingest stage and the placement of the processing stages are all design decisions. **b**, Each node of the traversal is a partially specified architecture. It is discarded when an optimistic bound on a constrained quantity already violates its limit (red), or when its optimistic objective vector is dominated by an architecture that has already been evaluated (red); only designs that survive to a leaf reach the expensive evaluator (blue).

Table 1: **Positioning with respect to the closest prior work.** “Partial-design pruning” denotes discarding incomplete designs before completing and evaluating them; “contention-aware” denotes objectives that depend on resource utilisation rather than accumulating over decisions.

Work	Year	System class	Obj.	Exact front	Partial-design pruning	Contention-aware	Real measurements
Blickle et al.[10]	1998	embedded synthesis	2	no	no	no	no
Lukasiewicz et al.[5]	2008	embedded synthesis	n	no	SAT-based	no	no
Neubauer et al.[4]	2018	heterogeneous MPSoC	3	yes	symbolic	no	no
Kouloumpris et al.[8]	2024	edge/hub/cloud	1	yes (BILP)	solver	no	UAV case
Sedghani et al.[11]	2024	computing continuum	1	no	no	partly	yes
CompDSE[1]	2025	industrial dCPS	perf.	—	none	yes	traces
Kouloumpris et al.[9]	2026	edge/hub/cloud	2	yes (BILP)	solver	no	case study
This work	2026	heterogeneous edge-cloud CPS	4	yes	bounds	yes	yes

Results

Exploration by elimination

HCA-DSE traverses architectures as a tree of partial designs and applies three tests of increasing cost at every node (Fig. 1). A partial design is discarded when it is structurally ill-formed, when an optimistic bound on any constrained quantity already violates its limit, or when an optimistic bound on the whole objective vector is dominated by an architecture that has already been evaluated. Only designs that survive to a leaf are handed to the expensive evaluator. The bounds are admissible by construction: each additive term of an objective is relaxed independently over the domains still reachable, and the waiting term becomes exact once the determinants of tier utilisation have been assigned (Methods). Safety of the two bound-based rules and preservation of the exact front follow from this property; the statements and proofs are in Methods and Supplementary Note 1.

Expensive evaluations become the scarce resource that is actually spent

Across four design-space scales and three workload classes, the funnel removes the design space before it is evaluated rather than after (Fig. 2). On a space of 51 840 candidates the method performs 41–50 full evaluations, discarding 99.90–99.92% of the raw space beforehand; on 345 600 candidates it performs 51–55, a reduction of 99.98–99.99%. The composition differs by workload: for data-intensive sensing the feasibility bounds remove the largest share, whereas for telemetry dominance pruning does most of the work, reflecting which constraint binds in each regime.

The reduction is exact, not approximate. For every scale at which exhaustive enumeration is tractable and for every workload, the set of objective vectors returned by HCA-DSE is identical to the exhaustive Pareto front, with recall and precision of 1.0 (Fig. 3). Admissibility of the bounds was verified exhaustively — all partial designs against all their completions — and for every pruned partial design all completions were enumerated and checked against the claim of the rule that discarded it. An independent exact solver built on a finite table and an SMT backend recovers the same front in all six small and medium cases, at the price of evaluating the entire structurally valid space first: 1080 and 3888 evaluator calls respectively (Supplementary Table S5).

The saving converts into time exactly when evaluation is expensive

Whether fewer evaluations translate into less time depends on what an evaluation costs. We measured the overhead of the exploration itself and the cost of one design-point evaluation for two evaluators, and projected the resulting speed-up (Fig. 4). With a closed-form analytical evaluator, whose cost is comparable to that of the bounds, the measured speed-up over exhaustive enumeration is only 1.32–3.15 $\times$ despite the 99.9% reduction in evaluations: in that regime pruning is not the right tool, and we report it as such. With a discrete-event evaluator measured at 32.8ms per design point, the same reduction projects to 86.4–146.7 $\times$ on the medium space and 638.2–771.1 $\times$ on the large one. The break-even evaluator cost — the price above which pruning repays its own overhead — is 1.25–2.55 μ s for the medium space and at or below 0.40 μ s for the large one, on the hardware and implementation used here. These are projections computed from measured counts and measured overheads; neither strategy was executed end to end with the discrete-event evaluator in the inner loop, and the break-even value is a property of this implementation rather than a constant of the method.

Guarantees at a fraction of a stochastic search budget

Under matched evaluation budgets, NSGA-II approaches the exact front but does not reach it, and cannot certify what is missing (Fig. 5, Table 2). At a budget of 1000 full evaluations on the large space its median Pareto recall is 0.500–0.741 with a hypervolume ratio of 0.946–0.984, while random search recovers at most 3.2% of the front points. HCA-DSE returns the complete front with 41–50 evaluations, between 20 and 24 times fewer. On the medium space NSGA-II is closer in hypervolume (0.999–1.000) yet still misses 7–20% of the front points. Across the six exploratory comparisons between NSGA-II and random search at the two largest budgets, none reaches $p < 0.05$ (unadjusted, 0.057–0.338; Cliff’s δ 0.080–0.295), so we do not claim a separation between the two stochastic baselines — the comparison that matters here is between guaranteed and approximate outcomes, not between two approximations.

The mechanism scales because the space is never built

Sweeping the raw space over five orders of magnitude, from 1.3×10^3 to 8.3×10^7 candidates, the number of expanded partial designs never exceeds 16 780 and the number of full evaluations stays between 18 and 61 (Fig. 6). The largest observed search time is 3.98s with memory tracing enabled, and peak traced allocation is 99.6 KiB, because the Cartesian product is generated lazily and never materialised. Exhaustive front equality cannot be checked at these sizes and is not claimed there.

Two internal ablations show where the reduction comes from. Removing the bound layers one at a time on the large space leaves 34 560 evaluations for structural pruning alone, 7025–12 700 with feasibility bounds, 322–415 with dominance bounds, and 41–50 with both — the layers are complementary, because feasibility pruning removes exactly the sub-trees in which the dominance test would otherwise be repeated (Supplementary Table S6). Toggling the contention-aware term of the bound while holding the traversal order fixed isolates its contribution: 3777–5139 incomplete designs carry a positive queueing contribution, of which 163–419 are discarded only because that term is present. All 36 order/toggle/scale/workload combinations preserve the exact front (Supplementary Table S7).

The ordering transfers to a real deployment

To test whether an architecture ranking produced at design time survives contact with a real system, we deployed the same architecture template as containers — one CPU per tier node, per-node link shaping with delay and bandwidth of the design under test, and a wide-area profile on the cloud hop — and measured 36 architectures, three repeats each, 108 runs (Table 3, Fig. 7). The analytical model was frozen before this round: it was not refitted to these observations.

Absolute prediction error is 25.4% (pooled MAPE) with a median absolute error of 6.51 ms. The decision-relevant quantity is different: among architecture pairs that the model separates by more than 25%, the measured order agrees with the predicted order for 88.6–91.3% of pairs within a workload. Pooling workloads raises this to 97.8% over 498 pairs, but part of that value comes from the separation between workloads rather than from resolution within one, so we base the claim on the within-workload figures. Six configurations were not reproducible across repeats, with a ratio of largest to smallest repeated mean latency of at least 1.5; they are retained in the summary rather than removed, and all sit close to saturation, where neither the analytical model nor the deployment itself is stable.

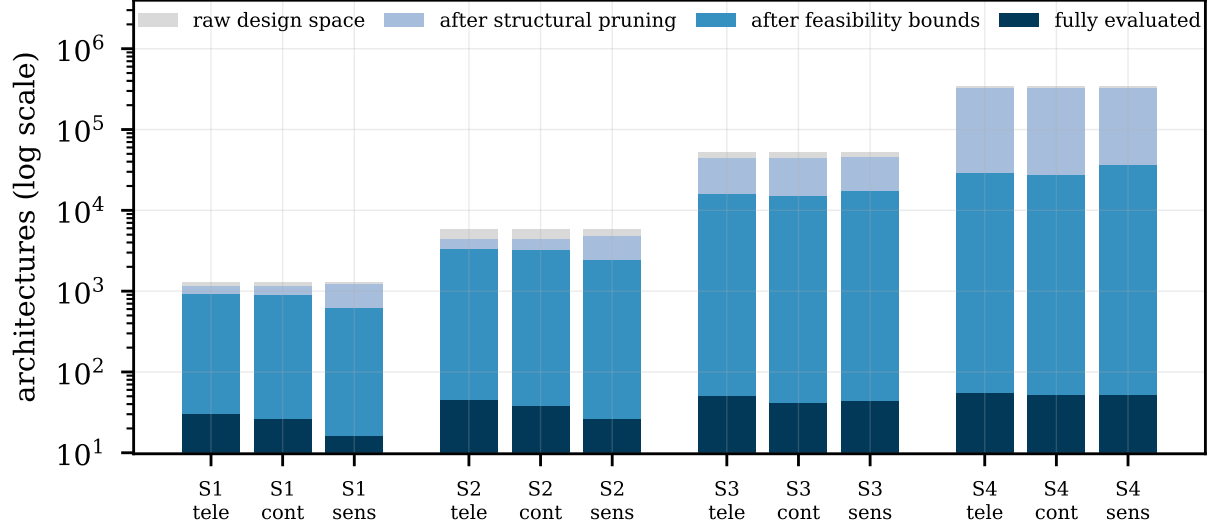


Figure 2: **Architectures are removed before they are evaluated.** Raw design space, survivors after structural and after bound-based elimination, and the number of architectures actually handed to the evaluator, for four design-space scales and three workload classes (logarithmic scale).

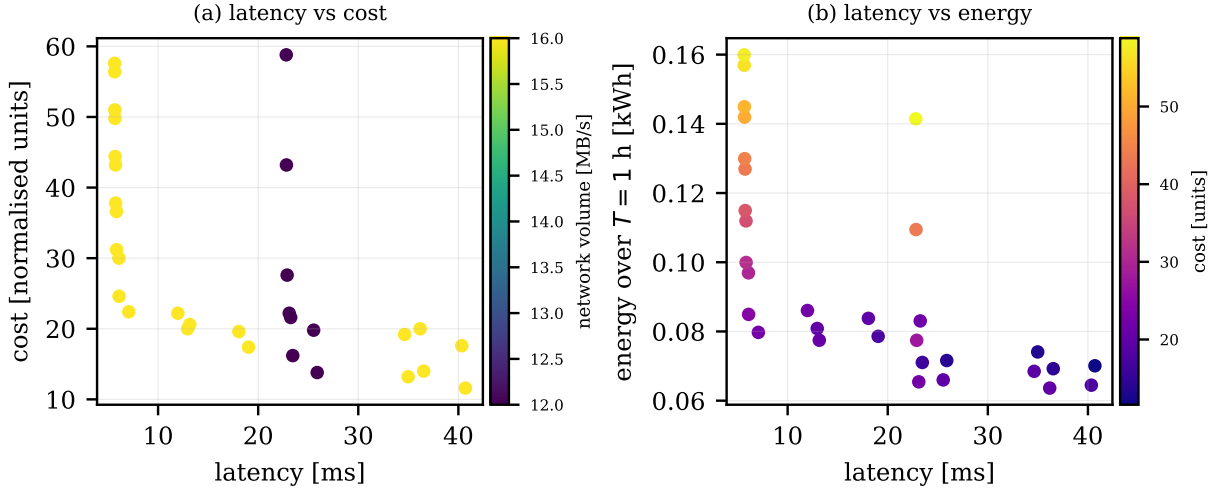


Figure 3: **The recovered set of optimal trade-offs.** Exact Pareto front on the 51 840-candidate space for the telemetry workload. **a**, latency versus cost, colour encodes network volume. **b**, latency versus energy over a one-hour horizon, colour encodes cost.

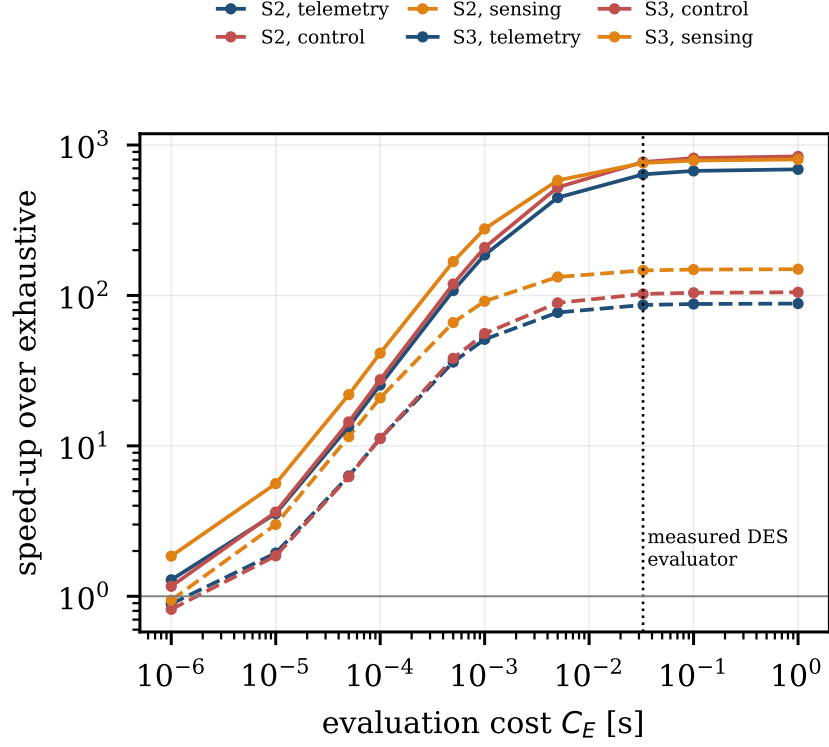


Figure 4: **The saving converts into time when evaluation is expensive.** Speed-up over exhaustive enumeration projected from measured evaluation counts and measured overheads as a function of the cost of one design-point evaluation. The dotted line marks the measured cost of the discrete-event evaluator.

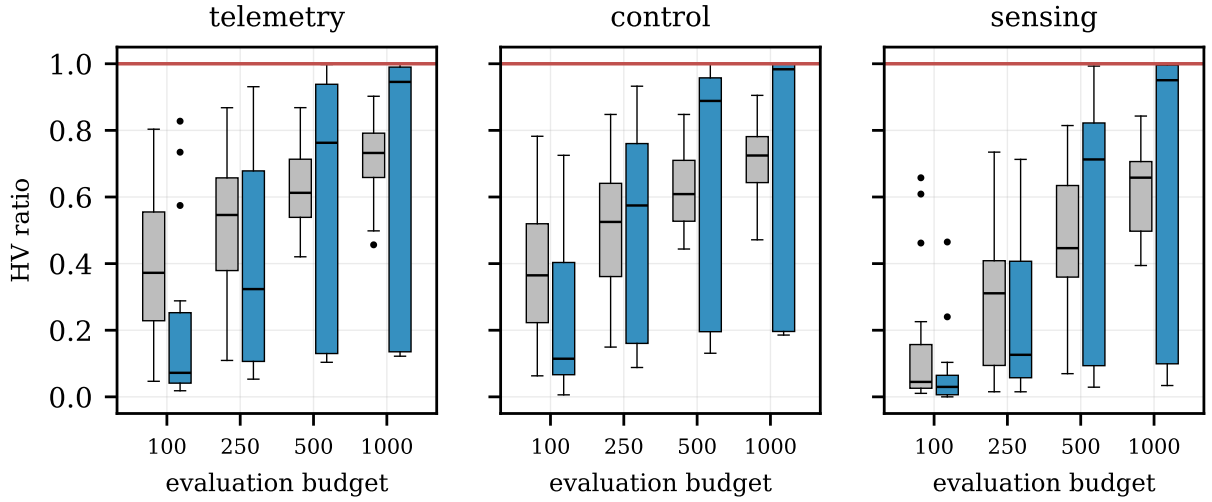


Figure 5: **Budget-matched stochastic exploration.** Hypervolume ratio against the exhaustive front on the 51840-candidate space over 20 seeds. Grey, random search; blue, NSGA-II. The red line marks the exact front recovered by the proposed method.

Table 2: **Budget-matched comparison.** Medians over 20 seeds at a budget of 1000 full evaluations; the full budget sweep is in Supplementary Tables S1–S3.

scale	workload	method	evals	recall	HV ratio	IGD ⁺	time [s]
S2	telemetry	exhaustive	3 888	1.000	1.000	0.0000	0.044
		random ($B=1000$)	1000	0.259	0.927	0.0388	0.014
		NSGA-II ($B=1000$)	1000	0.926	0.999	0.0002	0.147
		HCA-DSE	44	1.000	1.000	0.0000	0.036
S2	control	exhaustive	3 888	1.000	1.000	0.0000	0.042
		random ($B=1000$)	1000	0.240	0.922	0.0377	0.014
		NSGA-II ($B=1000$)	1000	0.800	0.999	0.0030	0.160
		HCA-DSE	37	1.000	1.000	0.0000	0.035
S2	sensing	exhaustive	3 888	1.000	1.000	0.0000	0.029
		random ($B=1000$)	1000	0.250	0.897	0.0626	0.010
		NSGA-II ($B=1000$)	1000	0.875	1.000	0.0017	0.182
		HCA-DSE	26	1.000	1.000	0.0000	0.016
S3	telemetry	exhaustive	34 560	1.000	1.000	0.0000	0.308
		random ($B=1000$)	1000	0.032	0.732	0.1147	0.011
		NSGA-II ($B=1000$)	1000	0.500	0.946	0.0454	0.145
		HCA-DSE	50	1.000	1.000	0.0000	0.138
S3	control	exhaustive	34 560	1.000	1.000	0.0000	0.265
		random ($B=1000$)	1000	0.000	0.725	0.1189	0.010
		NSGA-II ($B=1000$)	1000	0.648	0.984	0.0236	0.147
		HCA-DSE	41	1.000	1.000	0.0000	0.125
S3	sensing	exhaustive	34 560	1.000	1.000	0.0000	0.266
		random ($B=1000$)	1000	0.000	0.658	0.1569	0.010
		NSGA-II ($B=1000$)	1000	0.741	0.951	0.1677	0.175
		HCA-DSE	43	1.000	1.000	0.0000	0.085

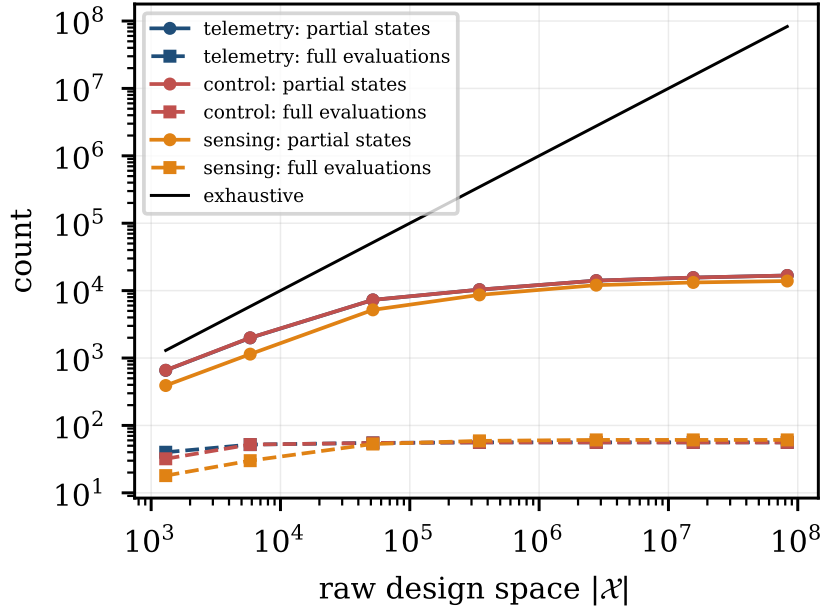


Figure 6: **The design space is never materialised.** Expanded partial designs and full evaluations against the raw design-space size, over five orders of magnitude.

Table 3: **Independent deployment validation.** Analytical predictions against measured run means over 36 architectures and 108 runs with the model frozen beforehand. Pair agreement is the share of architecture pairs whose measured order matches the predicted order among pairs separated by more than 25%. ALL pools workloads.

workload	n	MAPE [%]	Spearman ρ	agreement > 25%	pairs
telemetry	12	17.8	0.776	0.895	38
control	12	12.0	0.865	0.886	44
sensing	12	46.3	0.921	0.913	23
ALL	36	25.4	0.967	0.978	498

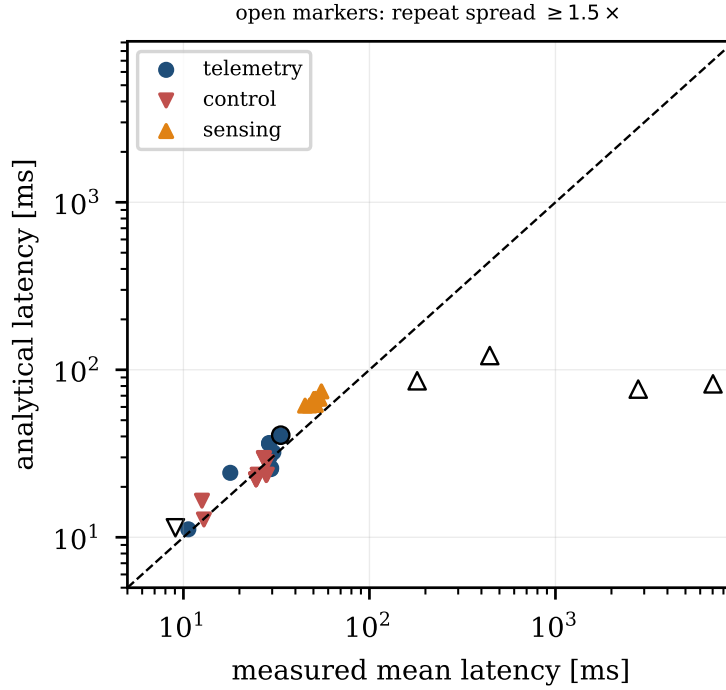


Figure 7: **Predicted against measured latency.** Analytical latency versus the median of three measured mean latencies on the independent containerised deployment. Open markers denote architectures whose repeated means differ by a factor of 1.5 or more.

Discussion

The practical obstacle to architecture exploration for distributed cyber-physical systems is the cost of an answer about one candidate, not the number of candidates. The results above change what that cost buys. Reducing the expensive evaluations from the size of the space to a few dozen, without giving up the guarantee that nothing optimal was discarded, turns a study that would require a simulator campaign into one that fits inside a design meeting; at the per-design-point costs reported for industrial trace-driven exploration[1] the projection is two to three orders of magnitude of wall-clock time.

What makes this possible is narrower than "pruning works", and worth stating precisely. Safe pruning of partial designs is an established idea in embedded system synthesis[4, 5] and in multi-objective search more generally[7, 6]. Its usual precondition — objectives that accumulate mono-

tonically with each decision — fails for the objective that dominates distributed cyber-physical systems, because latency depends on tier utilisation and utilisation is fixed jointly by several decisions. The contribution here is a way to keep exactness without that precondition: relax each additive term separately over the domains still reachable, and order the decisions so that the contention term stops being relaxed as early as possible. The mechanism is not specific to the template studied here. It applies wherever an objective decomposes into non-negative terms that are individually monotone in the free variables and where a subset of decisions determines a load-dependent term — which covers service placement in a computing continuum, sensing and control pipelines in robotics, and industrial edge deployments with mixed hardware tiers. Demonstrating that transfer on a structurally different template is the natural next step, and we make no claim of generality beyond the template evaluated here.

The exactness that HCA-DSE provides has a precise scope, and it is important not to overstate it. The guarantee is that the method returns the exact Pareto front of the analytical model; it says nothing about the model itself. A design-time method can be exact with respect to its own model and still inherit every error of that model. This is why the deployment round matters more than the correlation coefficient it produces: with 25% absolute error, the model is not a substitute for a simulator or a measurement, but it orders clearly separated architectures the same way the hardware does in roughly nine cases out of ten, which is what a screening stage has to do. The productive arrangement is therefore two-level. Analytical bounds remove the bulk of the space exactly and cheaply; the few dozen survivors are where a high-fidelity simulator or a physical deployment should spend its time. Nothing in the method requires the analytical model to be the final word, and an evaluator-centric methodology plugged in behind it would be strictly better than either component alone.

Three limits are worth stating plainly. First, when the evaluator is cheap the pruning overhead is comparable to the evaluation it avoids and the wall-clock benefit largely disappears — the measured speed-up with a closed-form evaluator is $1.32\text{--}3.15\times$ — so the method should be chosen on evaluator cost, not on space size. Second, the guarantee is only as good as the bounds: a bound that cannot be proven admissible must not be used, because an inadmissible bound silently converts an exact method into an approximate one with no signal in the output. Third, near saturation the design-time prediction breaks down and so does the deployment, as six of our measured configurations show; architectures in that regime should be treated as unresolved rather than ranked. Extending the guarantee to objectives that resist closed-form bounds, for example through surrogate bounds that remain provably admissible, is the most valuable open direction we see.

Methods

Architecture template and design space

The template has four tiers — devices, edge nodes, gateways and cloud — and three application stages per request. Pre-processing runs on the edge or gateway tier, aggregation always on the gateway tier, and analytics on the edge tier or in the cloud; which variant is instantiated is a design decision. A compute class is a tuple $(C, M, P^{\text{idle}}, P^{\text{peak}}, c)$ of capacity [MIPS], memory [MB], idle and peak power [W] and normalised cost; a link class is $(B, \tau^{\text{prop}}, e, c)$ of bandwidth [Mb/s], propagation delay, transfer energy [J/MB] and cost per attached node. The gateway-to-cloud connection is fixed infrastructure (200 Mb/s, 20 ms) and is not a design variable. An

architecture is

$$x = (n_e, \kappa_e, n_g, \kappa_g, \ell, \kappa_c, r, \pi), \quad (1)$$

with node counts n_e, n_g , classes $\kappa_e, \kappa_g, \kappa_c$, link class ℓ , replication factor r of the ingest stage and placement policy π . The design space is the Cartesian product $\mathcal{X} = D_1 \times \dots \times D_k$, $k = 8$. Four scales (S1–S4) and three workload classes — high-rate telemetry, deadline-dominated real-time control and payload-dominated sensing — are used throughout; their domains and parameters are listed in Supplementary Tables S8–S10.

Evaluation model

With $t(i)$ the tier executing stage i , N_t its node count and C_t the per-node capacity of its class, the offered load and utilisation are $\Theta_t = \sum_{i:t(i)=t} \Lambda w_i \mu_i$ and $\rho_t = \Theta_t / (N_t C_t)$, where Λ is the system arrival rate and $\mu_i = r$ for the ingest stage and 1 otherwise. A tier is modelled as one queueing station with the aggregate capacity of its nodes, deterministic service and Poisson arrivals, so that

$$L(x) = \sum_{i=1}^3 \frac{w_i}{C_{t(i)}} + \sum_{i=1}^3 \frac{w_i}{C_{t(i)}} \cdot \frac{\rho_{t(i)}}{2(1 - \rho_{t(i)})} + \sum_{(s,\ell) \in \mathcal{H}(x)} \left(\frac{8s}{B_\ell} + \tau_\ell^{\text{prop}} \right), \quad (2)$$

where $\mathcal{H}(x)$ is the set of hops implied by the policy and the replication factor. The middle term is the contention-dependent one. The aggregation is an approximation — a tier of N_t single-server nodes is an M/D/ c system rather than an M/D/1 system with c -fold capacity, and the approximation is optimistic — kept deliberately because it is the coarsest form that still makes latency depend on utilisation and remains monotone in the design variables. Both the discrete-event evaluator and the physical testbed implement per-node servers, so its cost is measured rather than assumed. Energy over a horizon $T = 1$ h combines idle and load-proportional compute power with transfer energy; cost sums node and link coefficients; network volume is the per-second byte flow. Feasibility requires $\rho_t \leq 0.85$ on every used tier, memory and bandwidth to fit, and the latency, cost and energy budgets to hold. Full expressions are in Supplementary Note 2.

Admissible bounds

Write $\mathcal{C}(p)$ for the completions of a partial design p . A bound LB_i is admissible if $LB_i(p) \leq f_i(x)$ for all $x \in \mathcal{C}(p)$. Two constructions are used.

Term-wise relaxation. If $f_i(x) = \sum_{t \in T_i} \varphi_t(x)$ with $\varphi_t \geq 0$ and $\underline{\varphi}_t(p) = \min_{x \in \mathcal{C}(p)} \varphi_t(x)$, then $LB_i(p) = \sum_t \underline{\varphi}_t(p)$ is admissible, because each term is bounded below independently. The minima are taken per term, so the combination realising them need not correspond to any completion; the bound is optimistic but valid. Every term of the model is monotone in each free variable, so each minimum is attained at a domain endpoint and is read from a table of precomputed extrema in constant time.

Exactness on determined terms. If the variables a term depends on are already assigned, the term is constant on $\mathcal{C}(p)$ and its exact value may replace the relaxed one, which preserves admissibility and tightens the bound. The decision order places the determinants of tier utilisation — node counts, classes, replication and policy — before the remaining variables, so the waiting term of (2) becomes exact early in the traversal instead of being relaxed to zero. This is what makes a contention-dependent objective usable without assignment monotonicity. If the determinants are fixed and $\rho_t \geq 1$, no completion has finite latency and the bound is ∞ , which is admissible.

Pruning rules and exactness

Three rules are applied at each node of the prefix tree. *Structural*: the well-formedness predicate σ (gateways do not outnumber edge nodes, replication does not exceed the edge count, replication is undefined for policies that place ingest off the edge tier) depends only on assigned variables and cannot be repaired by later assignments, so a violation eliminates all completions. It defines the search space rather than following from the constraints, and the exhaustive baseline applies the same predicate. *Feasibility*: if an admissible bound on a constrained quantity exceeds its limit, no completion is feasible. *Dominance*: if an evaluated feasible architecture y satisfies $F(y) \preceq LB(p)$, then every completion of p is dominated by y and none can be Pareto-optimal. Together with admissibility these give front preservation: the non-dominated set returned equals the exact Pareto front of the model in objective space. Proofs are in Supplementary Note 1. Two consequences are used in the experiments: the guarantee is order-independent, so traversal order affects efficiency only; and it is stated in objective space, so architectures sharing an objective vector may be represented once.

Verification of correctness

Theoretical correctness and implementation correctness are checked separately. Bound admissibility is verified exhaustively on the smallest scale — every partial design against every completion, for all three workloads. For every partial design that the implementation prunes, all of its completions are enumerated and checked against the claim of the rule that discarded it. Equality with the exhaustive front is re-checked on S1–S3 for every pruning variant as a regression test.

Baselines

Exhaustive enumeration provides ground truth on S1–S3. Random search and NSGA-II[3] (pymoo[12], population 40, SBX crossover, polynomial mutation, integer repair) run at matched budgets of 100, 250, 500 and 1000 full evaluations with 20 seeds each; budgets are charged to actual evaluator calls, including infeasible ones. An independent exact baseline encodes the finite evaluated relation and solves for the non-dominated set with an SMT solver[13]; it requires the full table to be built first and therefore consumes one evaluator call per structurally valid candidate. Recall and precision are computed on sets of objective vectors, hypervolume[14] is normalised by the exhaustive front, and IGD^+ [15] uses that front as reference, following the guidance on indicator choice in ref. 16. Stochastic baselines are compared with the one-sided Mann–Whitney U test[17] and Cliff’s δ [18], reported unadjusted and treated as exploratory.

Discrete-event evaluator

A higher-fidelity evaluator simulates each design with multi-server tiers, explicit link contention, deterministic service and Poisson arrivals. It serves two purposes: as an internal reference for the accuracy of the analytical model, and as the anchor of the evaluation-cost analysis, at a measured 32.8 ms per design point averaged over 40 designs.

Containerised deployment

Each edge node, gateway and cloud instance is a separate container limited to one CPU, so that a node behaves as the single server the model assumes at node level. Services are written in Go and share one binary: a request carries a plan of the remaining stages, each node consumes CPU work equal to w_i/C_t and forwards the remainder with the real payload to the next hop. Links are shaped

per node with `tc netem` [19] using the delay and rate of the link class under test, with a wide-area profile on the cloud hop; shaping is applied in the forward direction of each hop, matching the one-way transfers counted by the model. Replication is emulated faithfully: r replicas execute the ingest stage in parallel and carry the synchronisation payload, and the remainder of the chain executes once. A load generator produces Poisson arrivals, 900 requests per run of which 150 are discarded as warm-up. Thirty-six architectures — Pareto-optimal, dominated and near-boundary — were selected subject to fitting on one machine (≤ 4 edge nodes, ≤ 2 gateways), and each was run three times. The analytical model was frozen before this round.

Reproducibility

The implementation is Python 3 with NumPy, SciPy and pymoo; the deployment services are Go under Docker Compose. Every figure and every numeric table in this paper, main and supplementary, is generated from the raw result files by a single script, so no number is transcribed by hand.

Data availability

All result files underlying the figures and tables — exploration funnels, budget sweeps with per-seed metrics, ablations, scalability sweeps, discrete-event comparisons and the 108 measured deployment runs — are included in the replication package accompanying this submission.

Code availability

The exploration method, the analytical and discrete-event evaluators, the baselines, the containerised deployment and the verification tests are included in the same package, together with the single script that regenerates every display item from the raw results.

Author contributions

N.T. designed the method, implemented the software and the deployment, ran the experiments and wrote the manuscript. V.T. contributed to the system model and the experimental design and reviewed the manuscript.

Competing interests

The authors declare no competing interests.

References

- [1] F. S. Saadatmand, T. Stefanov, I. González Alonso, A. D. Pimentel, B. Akesson, CompDSE: A methodology for design space exploration of computing subsystems within complex cyber-physical systems, *IET Cyber-Physical Systems: Theory & Applications* 10 (1) (2025) e70019. doi:10.1049/cps2.70019.
- [2] M. Herget, F. S. Saadatmand, M. Bor, I. González Alonso, T. Stefanov, B. Akesson, A. D. Pimentel, Design space exploration for distributed cyber-physical systems: State-of-the-art, challenges, and directions, in: *25th Euromicro Conference on Digital System Design (DSD)*, 2022, pp. 632–640. doi:10.1109/DSD57027.2022.00090.
- [3] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Transactions on Evolutionary Computation* 6 (2) (2002) 182–197. doi:10.1109/4235.996017.

- [4] K. Neubauer, P. Wanko, T. Schaub, C. Haubelt, Exact multi-objective design space exploration using ASPmT, in: Design, Automation & Test in Europe Conference (DATE), 2018, pp. 257–260. doi:10.23919/DATE.2018.8342014.
- [5] M. Lukasiewicz, M. Glaß, C. Haubelt, J. Teich, Efficient symbolic multi-objective design space exploration, in: Asia and South Pacific Design Automation Conference (ASP-DAC), 2008, pp. 691–696. doi:10.1109/ASP-DAC.2008.4484040.
- [6] S. Petchrompo, D. W. Coit, A. Brintrup, A. Wannakrairot, A. K. Parlikad, A review of Pareto pruning methods for multi-objective optimization, Computers & Industrial Engineering 167 (2022) 108022. doi:10.1016/j.cie.2022.108022.
- [7] L. Mandow, J. L. Pérez-de-la Cruz, Multiobjective A* search with consistent heuristics, Journal of the ACM 57 (5) (2010) 1–25. doi:10.1145/1754399.1754400.
- [8] A. Kouloumpis, G. L. Stavrinides, M. K. Michael, T. Theocharides, An optimization framework for task allocation in the edge/hub/cloud paradigm, Future Generation Computer Systems 155 (2024) 354–366. doi:10.1016/j.future.2024.02.005.
- [9] A. Kouloumpis, G. L. Stavrinides, M. K. Michael, T. Theocharides, A reliability- and latency-driven task allocation framework for workflow applications in the edge–hub–cloud continuum, Future Generation Computer Systems 180 (2026) 108414. doi:10.1016/j.future.2026.108414.
- [10] T. Blickle, J. Teich, L. Thiele, System-level synthesis using evolutionary algorithms, Design Automation for Embedded Systems 3 (1) (1998) 23–58. doi:10.1023/A:1008899229802.
- [11] H. Sedghani, F. Filippini, D. Ardagna, SPACE4AI-D: A design-time tool for AI applications resource selection in computing continua, IEEE Transactions on Services Computing 17 (6) (2024) 4324–4339. doi:10.1109/TSC.2024.3479935.
- [12] J. Blank, K. Deb, pymoo: Multi-objective optimization in Python, IEEE Access 8 (2020) 89497–89509. doi:10.1109/ACCESS.2020.2990567.
- [13] L. de Moura, N. Bjørner, Z3: An Efficient SMT Solver, in: Tools and Algorithms for the Construction and Analysis of Systems, Vol. 4963, 2008, pp. 337–340. doi:10.1007/978-3-540-78800-3_24.
- [14] E. Zitzler, L. Thiele, Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach, IEEE Transactions on Evolutionary Computation 3 (4) (1999) 257–271. doi:10.1109/4235.797969.
- [15] H. Ishibuchi, H. Masuda, Y. Tanigaki, Y. Nojima, Modified distance calculation in generational distance and inverted generational distance, in: Evolutionary Multi-Criterion Optimization, Springer, 2015, pp. 110–125. doi:10.1007/978-3-319-15892-1_8.
- [16] C. Audet, J. Bignon, D. Cartier, S. Le Digabel, L. Salomon, Performance indicators in multiobjective optimization, European Journal of Operational Research 292 (2) (2021) 397–422. doi:10.1016/j.ejor.2020.11.016.
- [17] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.
- [18] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychological Bulletin 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.
- [19] S. Hemminger, Network emulation with NetEm, in: Linux Conference Australia (linux.conf.au), 2005.